Prac1.1: Factorial of Trailing Zeroes

```
class Solution {
    public int trailingZeroes(int n) {
        int ans = 0;
        if (n < 5) return 0;
        while (n > 0) {
            n = n / 5;
            ans += n;
        }
        return ans;
    }
}
```

Prac1.2: Extended Euclidean Distance

```
class Solution {
public:
   int extendedGCD(int a, int b, int &x, int &y) {
      if (b == 0) {
         x = 1;
         y = 0;
         return a;
      }

      int x1, y1;
      int g = extendedGCD(b, a % b, x1, y1);

      x = y1;
      y = x1 - (a / b) * y1;

      return g;
   }

   int gcd(int a, int b) {
      int x, y;
      return extendedGCD(a, b, x, y);
   }
};
```

Prac1.3: Missing Number

```
class Solution {
public:
   int missingNumber(vector<int>& nums) {
      int xor_num = 0,xor2 = 0, n = nums.size();
      for(int i = 0; i <= n; i++)
      {
         xor_num = xor_num ^ i;
      }
      for(int i = 0; i < n; i++)
      {
         xor2 = xor2 ^ nums[i];
      }
      return xor_num ^ xor2;
   }
};
```

Prac1.4: Single Number 111

```
class Solution {
public:
   vector<int> singleNumber(vector<int>& nums) {
      long ext = 0;
      for (int i = 0; i < nums.size(); i++) {
         ext ^= nums[i];
      }
      long x = 0, y = 0;
      long diff = ext & (-ext);
      for (int i = 0; i < nums.size(); i++) {
         if (nums[i] & diff) {
            x ^= nums[i];
         } else {
            y ^= nums[i];
         }
      }
      return {(int)x, (int)y};
   }
};
```

Prac2.1: Subsets

```
class Solution {
public:
    vector<vector<int>> ans;
    void tra(vector<int>& nums, int i, vector<int> rst) {
        if (i >= nums.size()) {
            ans.push_back(rst);
            return;
        }
        tra(nums, i + 1, rst);
        rst.push_back(nums[i]);
        tra(nums, i + 1, rst);
    }
    vector<vector<int>> subsets(vector<int>& nums) {
        tra(nums, 0, {});
        return ans;
    }
};
```

Prac2.2: Permutations

```
class Solution {
public:
    vector<vector<int>> ans;
    vector<int> temp;
    void show(vector<bool>& visited, vector<int>& nums) {
        if (temp.size() == nums.size()) {
            ans.push_back(temp);
            return;
        }
        for (int i = 0; i < nums.size(); i++) {
            if (!visited[i]) {
                visited[i] = true;
                temp.push_back(nums[i]);
                show(visited, nums);
                visited[i] = false;
                temp.pop_back();
            }
        }
    }
    vector<vector<int>> permute(vector<int>& nums) {
        vector<bool> visited(nums.size(), false);
        show(visited, nums);
        return ans;
    }
};
```

Prac3.1: Asteroid Collision

```
class Solution {
public:
    vector<int> asteroidCollision(vector<int>& asteroids) {
        stack<int> st;
        for (int curr : asteroids) {
            bool alive = true;
            while (!st.empty() && curr < 0 && st.top() > 0) {
                int top = st.top();
                if (abs(top) < abs(curr)) {
                    st.pop();
                    continue;
                }
                if (abs(top) == abs(curr)) {
                    st.pop();
                    alive = false;
                    break;
                }
                alive = false;
                break;
            }
            if (alive) {
                st.push(curr);
            }
        }
        vector<int> ans(st.size());
        for (int i = ans.size() - 1; i >= 0; i--) {
            ans[i] = st.top();
            st.pop();
```

```
        }
        return ans;
    }
};
```

Prac3.2: Trapping Rainwater

```
class Solution {
public:
    int trap(vector<int>& height) {
        int left = 0;
        int right = height.size() - 1;
        int leftMax = height[left];
        int rightMax = height[right];
        int water = 0;
        while (left < right) {
            if (leftMax < rightMax) {
                left++;
                leftMax = max(leftMax, height[left]);
                water += leftMax - height[left];
            } else {
                right--;
                rightMax = max(rightMax, height[right]);
                water += rightMax - height[right];
            }
        }
        return water;
    }
};
```

Prac4.1: Sort Vowels in a String

```
class Solution {
public:
    string sortVowels(string s) {
        vector<int> freq(128, 0);
        vector<bool> isVowel(128, false);
        string vowels = "aeiouAEIOU";
        for (char v : vowels) {
            isVowel[v] = true;
        }
        for (char c : s) {
            if (isVowel[c]) {
                freq[c]++;
            }
        }
        string order = "AEIOUaeiou";
        int pos = 0;
        for (char &c : s) {
            if (isVowel[c]) {
                while (pos < order.size() && freq[order[pos]] == 0) {
                    pos++;
                }
                c = order[pos];
                freq[order[pos]]--;
            }
        }
        return s;
    }
```

```
};
```

Prac4.2: Wildcard Matching

```
class Solution {
public:
    bool isMatch(string s, string p) {
        int i = 0, j = 0;
        int star = -1;
        int match = 0;
        while (i < s.size()) {
            if (j < p.size() && (p[j] == '?' || p[j] == s[i])) {
                i++;
                j++;
            }
            else if (j < p.size() && p[j] == '*') {
                star = j;
                match = i;
                j++;
            }
            else if (star != -1) {
                j = star + 1;
                match++;
                i = match;
            }
            else {
                return false;
            }
        }
```

```
      while (j < p.size() && p[j] == '*')
        j++;
      return j == p.size();
    }
};
```

Prac4.3: Longest Awesome Substring

```
class Solution {
public:
int longestAwesome(string s) {
    vector<int> dp(1024, s.size());
    int res = 0, mask = 0;
    dp[0] = -1;
    for (auto i = 0; i < s.size(); ++i) {
      mask ^= 1 << (s[i] - '0');
      res = max(res, i - dp[mask]);
      for (auto j = 0; j <= 9; ++j)
        res = max(res, i - dp[mask ^ (1 << j)]);
      dp[mask] = min(dp[mask], i);
    }
    return res;
}
};
```

Prac5.1: Daily Temperatures

```
class Solution {
public:
    vector<int> dailyTemperatures(vector<int>& T) {
        vector<int> ans(size(T));
        for(int n = size(T), cur = n-2; cur >= 0; cur--) {
            int next = cur + 1;
            while(next < n && T[next] <= T[cur])
                next += ans[next] ? ans[next] : n;
            if(next < n) ans[cur] = next-cur;
        }
        return ans;
    }
};
```

Prac5.2: Defuse the Bomb

```
class Solution {
    public int[] decrypt(int[] code, int k) {
        int[] result = new int[code.length];
        if (k == 0) return result;
        int start = 1, end = k, sum = 0;
        if (k < 0) {
            start = code.length - Math.abs(k);
            end = code.length - 1;
        }
        for (int i = start; i <= end; i++) sum += code[i];
        for (int i = 0; i < code.length; i++) {
            result[i] = sum;
            sum -= code[(start) % code.length];
            sum += code[(end + 1) % code.length];
            start++;
            end++;
        }
        return result;
    }
}
```

Prac6.1: Number of Provinces

```
class Solution {
    public void dfs(int node, int[][] isConnected, boolean[] visit) {
        visit[node] = true;
        for (int i = 0; i < isConnected.length; i++) {
            if (isConnected[node][i] == 1 && !visit[i]) {
                dfs(i, isConnected, visit);
            }
        }
    }
    public int findCircleNum(int[][] isConnected) {
        int n = isConnected.length;
        int numberOfComponents = 0;
        boolean[] visit = new boolean[n];
        for (int i = 0; i < n; i++) {
            if (!visit[i]) {
                numberOfComponents++;
                dfs(i, isConnected, visit);
            }
        }
        return numberOfComponents;
    }
}
```

Prac6.2: Course Schedule

```
class Solution {
private:
  bool dfs(int node, vector<int>& vis, vector<int>& pathVis,
        vector<int> adj[]) {
    vis[node] = 1;
    pathVis[node] = 1;
    for (auto adjNode : adj[node]) {
      if (!vis[adjNode]) {
        if (dfs(adjNode, vis, pathVis, adj) == true)
          return true;
      }
      else if (pathVis[adjNode]) {
        return true;
      }
    }
    pathVis[node] = 0;
    return false;
  }
public:
  bool canFinish(int numCourses,
            vector<vector<int>>& prerequisites) {
    vector<int> adj[numCourses];
    for (int i = 0; i < prerequisites.size(); i++) {
      int u = prerequisites[i][1];
      int v = prerequisites[i][0];
      adj[u].push_back(v);
```

```
    }
    vector<int> vis(numCourses, 0);
    vector<int> pathVis(numCourses, 0);
    for (int i = 0; i < numCourses; i++) {
      if (!vis[i]) {
        if (dfs(i, vis, pathVis, adj) == true)
          return false;
      }
    }
    return true;
  }
};
```

Prac7.1: Champagne Tower

```
class Solution {
  public double champagneTower(int poured, int query_row, int query_glass) {
    double[] dp = new double[query_row + 1];
    dp[0] = poured;
    for (int i = 0; i < query_row; i++) {
      for (int j = i; j >= 0; j--) {
        double excess = (dp[j] - 1.0) / 2.0;
        if (excess > 0) {
          dp[j + 1] += excess;
          dp[j] = excess;
        } else {
          dp[j] = 0;
        }
      }
```

```
      }
      return Math.min(1.0, dp[query_glass]);
    }
}
```

Prac7.2: House Robber

```
class Solution {
public:
    int rob(vector<int>& A) {
        int prev2 = 0, prev = 0, cur = 0;
        for(auto i : A) {
            cur = max(prev, i + prev2);
            prev2 = prev;
            prev = cur;
        }
        return cur;
    }
};
```